Capstone Project Submission Guidelines

Deliverables, structure and documentation requirements

1. Submission Overview

Participants are required to submit all project deliverables through a GitHub repository. The repository should contain the complete source code, documentation, architecture artifacts, presentation materials, and supporting files required to evaluate the solution.

The submission should demonstrate an end-to-end working GenAI solution aligned with the selected capstone project. Local execution of the solution is sufficient for evaluation.

2. Submission Deliverables

Mandatory Deliverables

1. Architecture Diagram (PDF/JPEG)
2. Design Document (PDF)
3. Complete Source Code
4. README File
5. API Documentation
6. Dataset Details Document
7. Presentation Deck (PPT/PDF)

Recommended Deliverables

- Evaluation Reports (DeepEval / RAGAS)
- Sample Test Cases

3. GitHub Repository Naming Convention

Use the following repository naming format: <Name of the Participant><Project Domain Name>

Example: Veena_Telecom

Repository names should:

- Use only letters, numbers, and underscores
- Avoid spaces
- Clearly indicate the selected project

4. Project Folder Structure

```
ProjectRepository/
├── architecture/
│   └── Architecture_Diagram.pdf
├── design/
│   └── Design_Document.pdf
├── src/
│   ├── api/
│   ├── agents/
│   ├── rag/
│   ├── services/
│   ├── utils/
│   └── main.py
├── data/
```

```
│   └── sample_data/
├── docs/
│   ├── API_Documentation.pdf
│   └── Evaluation_Report.pdf
├── tests/
├── presentation/
│   └── Project_Presentation.pptx
├── requirements.txt
├── README.md
└── .gitignore
```

5. File Naming Conventions

- Architecture_Diagram.pdf
- Design_Document.pdf
- API_Documentation.pdf
- Evaluation_Report.pdf
- Project_Presentation.pptx
- Dataset_Details.pdf
- README.md

Use meaningful names and avoid spaces in file names.

6. README Requirements

- Project Overview
- Problem Statement
- Business Objective
- Solution Summary
- Technology Stack
- Programming Language
- LLM Used
- Embedding Model
- Vector Database
- Frameworks Used
- Installation Instructions
- Environment Setup
- Dependency Installation
- API Key Configuration
- Local Deployment Instructions
- Sample Queries

Include 3–5 sample user queries and expected outputs.

7. Design Document Requirements

- **Problem Statement** – Business problem being addressed.
- **Architecture Design** – High-level architecture diagram and explanation.
- **Technology Selection** – Justification for LLM, Embedding Model, Vector Database and Frameworks.
- **Retrieval Strategy** – Chunking Strategy, Embedding Approach, Retrieval Method, Hybrid Search (if implemented).

- **Agent Design** – Agent Roles, Workflow and Agent Communication.
- **Evaluation Strategy** – DeepEval, RAGAS, Test Cases and Metrics Used.
- **Challenges and Trade-offs** – Design decisions and limitations.

8. Architecture Diagram Requirements

The architecture diagram should clearly show:

Dataset → Preprocessing → Chunking → Embeddings → Vector Database → Retriever → LLM / RAG Pipeline → Agents → API Layer → User Interface

The diagram should be readable and professionally presented.

9. API Documentation Requirements

Document all APIs exposed by the solution. For each API include:

- Endpoint
- HTTP Method
- Request Format
- Response Format
- Sample Request
- Sample Response

Example: POST /search-incidents

10. Dataset Documentation

- Dataset Name
- Dataset Source
- Dataset Link
- Data Format
- Key Fields
- Dataset Purpose
- Data Preprocessing Performed

11. Local Deployment Requirements

Projects should be runnable locally using the documented Python environment. Docker, Azure deployment, Kubernetes and CI/CD pipelines are not required.

- Provide complete dependency information in requirements.txt.
- Provide environment setup and API-key configuration instructions in README.md.
- Run the backend using FastAPI/Uvicorn.
- Run the front end using Streamlit.
- Ensure the API and UI can be demonstrated end-to-end on the student's local machine.
- Do not commit API keys, passwords, or other secrets to GitHub.

Typical local commands:

```
uvicorn app.main:app --reload
streamlit run frontend.py
```

12. Presentation Requirements

1. Problem Statement
2. Business Need
3. Solution Overview
4. Architecture
5. Data Flow
6. Technology Stack
7. Agent Workflow
8. Demo
9. Evaluation Results
10. Challenges and Learnings

Presentation duration: 10 minutes.

13. Evaluation Artifacts (Recommended)

- Retrieval Results
- DeepEval Scores
- RAGAS Scores
- Test Cases
- Sample Outputs

These artifacts strengthen the submission and improve evaluation quality.

14. Submission Process

1. Create GitHub Repository - Prepare files, Compress as ZIP.
2. Upload all deliverables to Github.
3. Verify repository access permissions.
4. Share the Team / Project Name, GitHub Repository URL, and Team Member Details with the organizers.

15. Submission Checklist

- ☐ Source code is complete
- ☐ README is updated
- ☐ Architecture diagram included
- ☐ Design document included
- ☐ Presentation deck included
- ☐ APIs are working
- ☐ Sample data included
- ☐ Repository is accessible
- ☐ Team details are provided
- ☐ All deliverables follow naming conventions
- ☐ Local deployment instructions are included
- ☐ No secrets or API keys are committed

16. Important Note

Docker, Azure deployment, Kubernetes, and CI/CD pipelines are optional. Projects will be evaluated based on functionality, architecture, RAG implementation, agent workflows, documentation, local

execution, and presentation. Local execution of the solution is sufficient for evaluation.